01

설계 목표

CNN의 Activation function은 ReLU
마지막 출력 레이어는 Softmax
나머지 신경망의 Activation function은 Sigmoid
3개 이상의 hidden-Layer
Back-propagation 은 Cross Entropy
Pooling 은 2x2 Max Pooling

전체 시스템

입력 : 28x28 image 3개 (0~9 / 0~9 / + , - , X)
출력 : 14bit 이진수 (7segment 2개)

INPUT	image 3개	23bit one-hot-encoding	8bit 이진수
	CNN (0~9)	**NN** (0~9)	**7segment**
OUTPUT	23bit one-hot-encoding	8bit 이진수	14bit 이진수

CNN

입력 : 28x28 image 3개
출력 : 23bit one-hot-encoding

image
(0~9)

image
(0~9)

Conv

ReLU

Pool

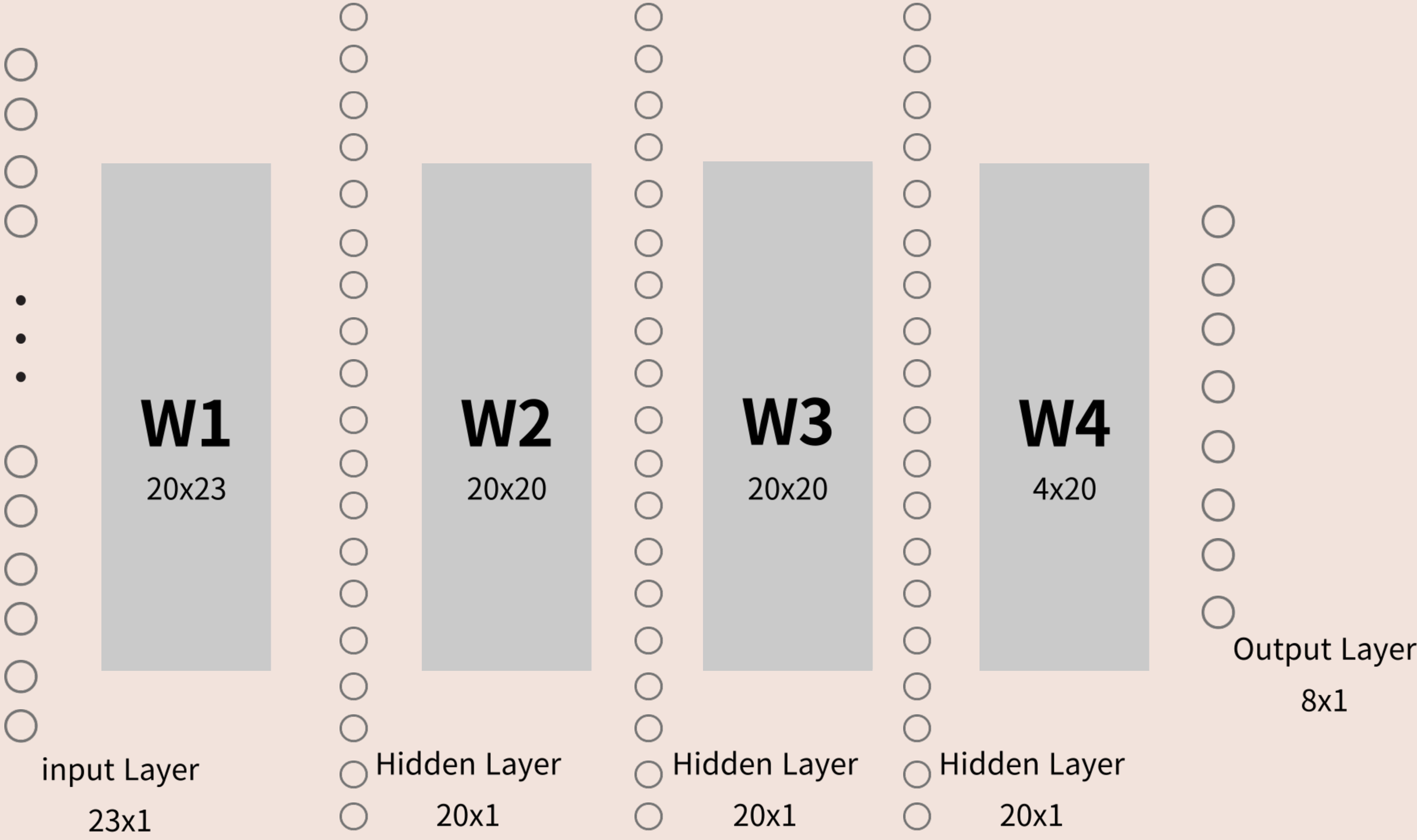
ReLU

image
(+,-,X)

softmax

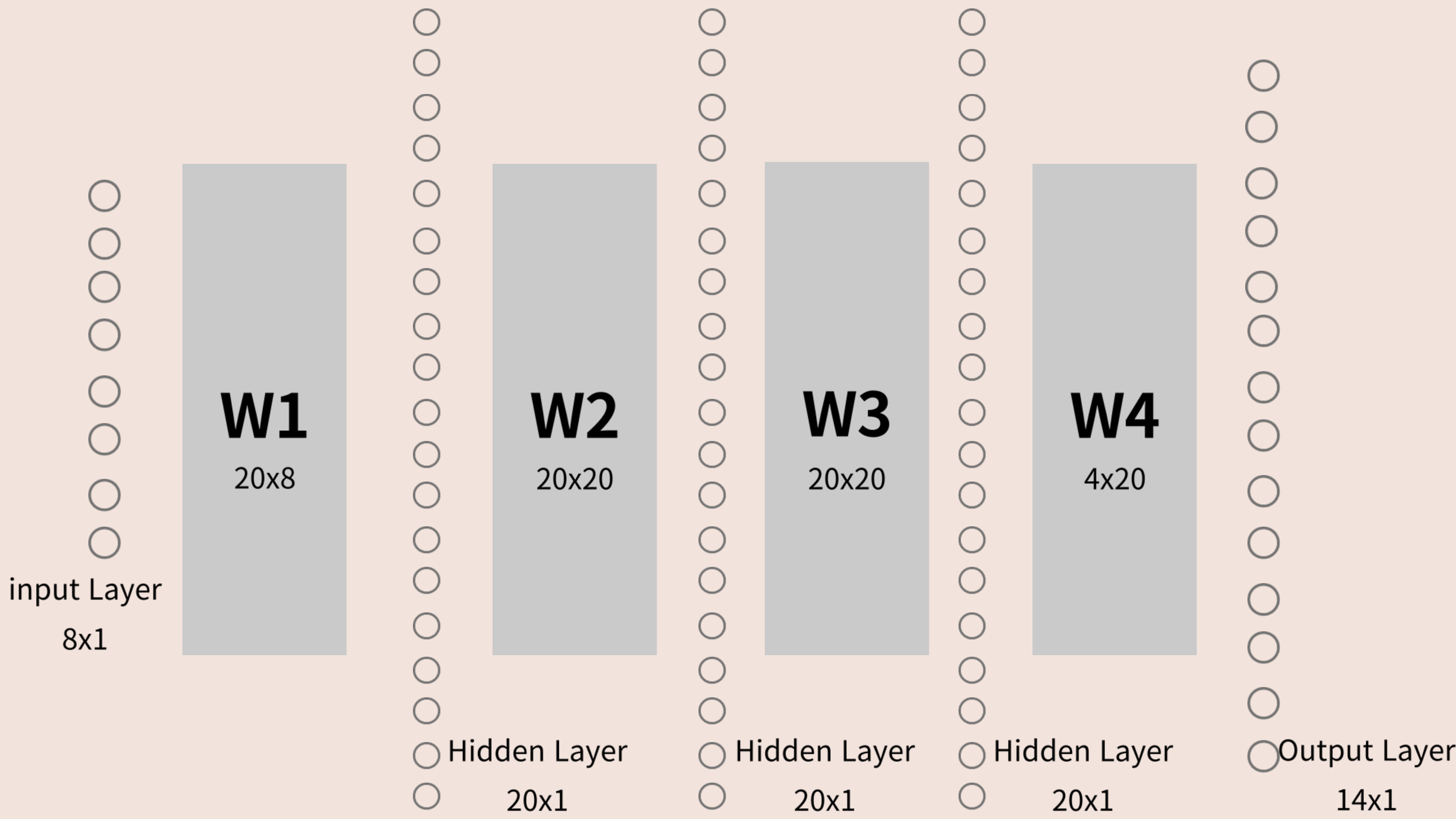
NN

입력 : 23bit one-hot-encoding
출력 : 8bit 이진수


W1
20x23
W2
20x20
W3
20x20
W4
4x20
input Layer
23x1
Hidden Layer
20x1
Hidden Layer
20x1
Hidden Layer
20x1
Output Layer
8x1

7segment

입력 : 8bit 이진수
출력 : 14bit 이진수 (7segment 2개)


W1
20x8
W2
20x20
W3
20x20
W4
4x20
input Layer
8x1
Hidden Layer
20x1
Hidden Layer
20x1
Hidden Layer
20x1
Output Layer
14x1

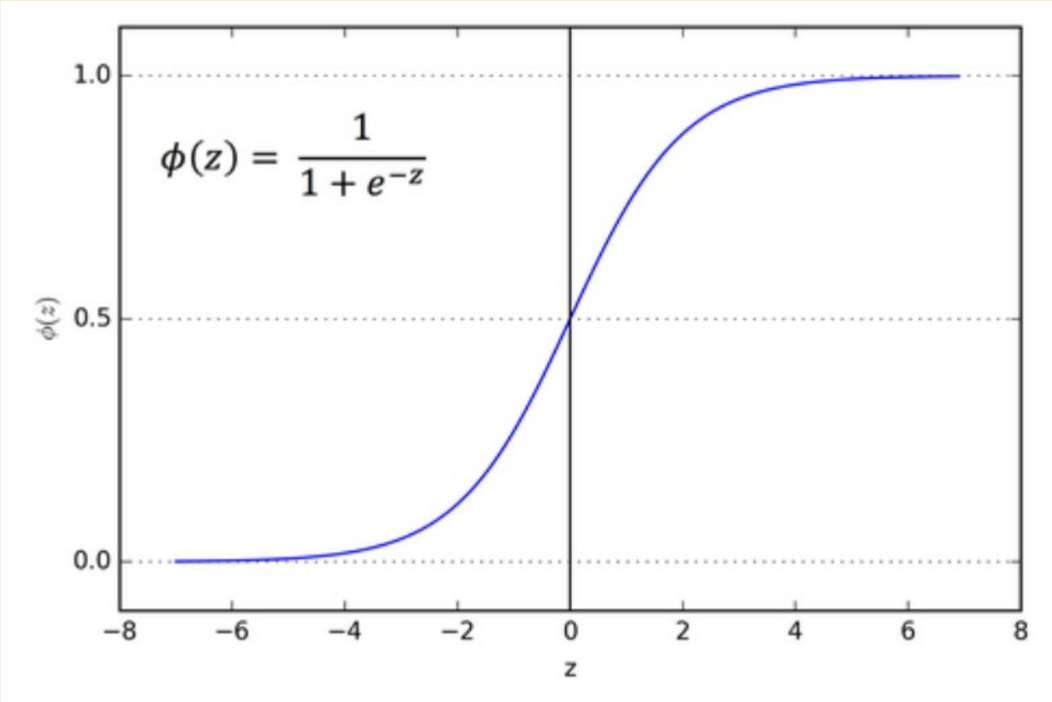
02

사용된 함수

sigmoid function
ReLU function
Conv function
Pool function
softmax function
MnistConv funtion
BackpropCE function

+9 Sigmoid.m Conv.m

```
function y = Sigmoid(x)
  y = 1 ./ (1 + exp(-x));
end
```

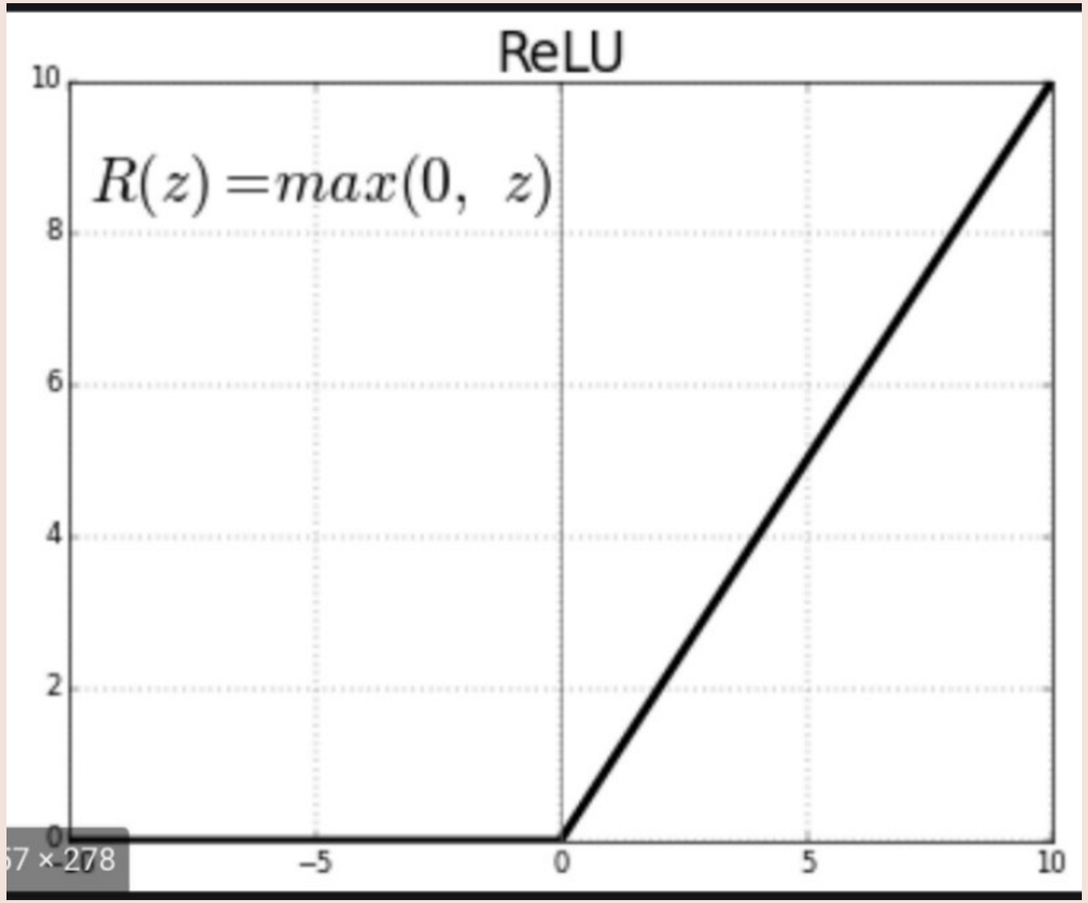


Sigmoid

Sigmoid function을 Matlab 으로 구성한 것이다.

+9 ReLU.m Pool.

```
function y = ReLU(x)
  y = max(0, x);
end
```



ReLU

ReLU function을 Matlab으로 구성한 것이다.

+9 | Sigmoid.m | Conv.m * | ReLU.m | Pool.m

```
function y = Conv(x, W)

[wrow, wcol, numFilters] = size(W);      % 9x9x20
[xrow, xcol, ~         ] = size(x);      % 28x28

yrow = xrow - wrow + 1;                  % 28 - 9 + 1 = 20
ycol = xcol - wcol + 1;                  % 28 - 9 + 1 = 20

y = zeros(yrow, ycol, numFilters);       % 20x20x20

for k = 1:numFilters
  filter = W(:, :, k);                   % 9x9x1
  filter = rot90(squeeze(filter), 2);    % 180도 회전
  y(:, :, k) = conv2(x, filter, 'valid');
end

end
```

Conv

convolution은 input Data를 늘리는 용도로 사용되며, 특징점을 도출하는 용도로 사용된다.

+9 | ReLU.m | Pool.m | Softmax.m | loadMNISTImages.m | loadM

```
function y = MaxPool(x)
%
% 2x2 mean pooling
%
%
[xrow, xcol, numFilters] = size(x);     %20x20x20

y = zeros(xrow/2, xcol/2, numFilters);  %10x10x20
ym = [];
ym1 =[];
for k = 1:numFilters                    %1~20
    for m = 1:2:xrow-1                  %1:2:20-1 -> 1:2:19 (1, 3, 5, ... , 19)
        for n = 1:2:xcol-1              %1:2:20-1 -> 1:2:19 (1, 3, 5, ... , 19)
            max_y = max(max(max(x(m:m+1, n:n+1, k))));
            ym = [ym, max_y(1)];
        end
        ym1 = [ym1; ym];
        ym =[];
    end
    y(:,:,k) = ym1;
    ym1=[];
end
end
```

Pool

pooling은 convolution을 했을때 특징점이 도출되지않은 불필요한 데이터를 지우는데 사용된다.

이번 프로젝트에서는 maxpool을 사용하였다.

+9 Softmax.m loadM

```
function y = Softmax(x)
  ex = exp(x);
  y  = ex / sum(ex);
end
```

softmax

Sigmoid function을 Matlab 으로 구성한 것이다.

+9 | Softmax.m | loadMNISTImages.m | loadMNISTLabels.m | MnistConv1.m

```
function [W1, W5, Wo] = MnistConv1(W1, W5, Wo, X, D)
%
%

    alpha = 0.01;   %Learning rate
    beta  = 0.95;   %momentum

    momentum1 = zeros(size(W1));
    momentum5 = zeros(size(W5));
    momentumo = zeros(size(Wo));

    N = length(D);  % N = 8000

    bsize = 100;    %batch size
    blist = 1:bsize:(N-bsize+1);    %1:100:8000-100+1 -> 1:100:7901 (1,101,201, ... , 7901)

    % One epoch loop
    %
    for batch = 1:length(blist)     % 1 ~ 80
      dW1 = zeros(size(W1));
      dW5 = zeros(size(W5));
      dWo = zeros(size(Wo));

      % Mini-batch loop
      %
      begin = blist(batch);         % 1
      for k = begin:begin+bsize-1   % 1: 1+100-1 -> 1:100
        % Forward pass = inference
        %
        x  = X(:, :, k);              % Input,           28x28
        y1 = Conv(x, W1);             % Convolution,  20x20x20
        y2 = ReLU(y1);                % 20x20x20
        y3 = Pool(y2);                % Pooling,      10x10x20
        y4 = reshape(y3, [], 1);      % 2000x1
        v5 = W5*y4;                   % ReLU,         (100x2000) x (2000x1) -> 100x1
        y5 = ReLU(v5);                % 100x1
        v  = Wo*y5;                   % Softmax,      (10x100) x (100x1) -> 10x1
        v(isnan(v)) = 0;
```

MnistConv

MnistConv function을
Matlab으로 구성한 것이다.

```
        y  = Softmax(v);                % 10x1
        y(isnan(y)) = 0;

        % One-hot encoding
        %
        d = zeros(10, 1);               % 10x1
        d(sub2ind(size(d), D(k), 1)) = 1;   % k번째 위치에 1을 설정

        % Backpropagation
        %
        e      = d - y;                   % Output layer  10x1
        delta  = e;                         %Cross Entropy 10x1

        e5     = Wo' * delta;             % Hidden(ReLU) layer (100x10) x (10x1) -> (100x1)
        delta5 = (y5 > 0) .* e5;            %100x1

        e4     = W5' * delta5;            % Pooling layer 100x1

        e3     = reshape(e4, size(y3));     % 10x10x20

        e2 = zeros(size(y2));
        W3 = ones(size(y2)) / (2*2);
        for c = 1:20
          e2(:, :, c) = kron(e3(:, :, c), ones([2 2])) .* W3(:, :, c);
        end

        delta2 = (y2 > 0) .* e2;          % ReLU layer

        delta1_x = zeros(size(W1));       % Convolutional layer
        for c = 1:20
          delta1_x(:, :, c) = conv2(x(:, :), rot90(delta2(:, :, c), 2), 'valid');
        end

        dW1 = dW1 + delta1_x;
        dW5 = dW5 + delta5*y4';
        dWo = dWo + delta *y5';
      end

      % Update weights
      %
      dW1 = dW1 / bsize;
      dW5 = dW5 / bsize;
      dWo = dWo / bsize;

      momentum1 = alpha*dW1 + beta*momentum1;
      W1        = W1 + momentum1;

      momentum5 = alpha*dW5 + beta*momentum5;
      W5        = W5 + momentum5;

      momentumo = alpha*dWo + beta*momentumo;
      Wo        = Wo + momentumo;
    end

end
```

MnistConv

MnistConv function을
Matlab으로 구성한 것이다.

+9 CNN2.m MnistConv2.m CNN3.m BackpropCE1.m

```
function [W1, W2, W3, W4] = BackpropCE1(W1, W2, W3, W4, X, D)
  alpha = 0.2;          %Learning Rate
  N = 300;               %총 300개
  for k = 1:N
      %Input Set {input data, Correct Answer}
    x = X(k, :)';        % x = a column vector
    d = D(k, :)';

    %1st Hidden Layer
    v1 = W1*x;          %20x1 = 20x23 * 23x1
    y1 = Sigmoid(v1);   %20x1

    %2nd Hidden Layer
    v2 = W2*y1;         %20x1 = 20x20 * 20x1
    y2 = Sigmoid(v2);   %20x1

    %3rd Hidden Layer
    v3 = W3*y2;         %20x1 = 20x20 * 20x1
    y3 = Sigmoid(v3);   %20x1

    %Output Layer
    v  = W4*y3;         %8x1 = 8x20 * 20x1
    y  = Sigmoid(v);    %8x1
```

BackpropCE

Cross Entropy를 이용한 다층 신경망을 설계하는 함수이다.

```
    %Back-propagation
    e     = d - y;
    delta = e;

    e3     = W4'*delta;
    delta3 = y3.*(1-y3).*e3;

    e2     = W3'*delta3;
    delta2 = y2.*(1-y2).*e2;

    e1     = W2'*delta2;
    delta1 = y1.*(1-y1).*e1;

    dW1 = alpha*delta1*x';
    W1 = W1 + dW1;

    dW2 = alpha*delta2*y1';
    W2 = W2 + dW2;

    dW3 = alpha*delta3*y2';
    W3 = W3 + dW3;

    dW4 = alpha*delta*y3';
    W4 = W4 + dW4;
  end

end
```

BackpropCE

Cross Entropy를 이용한 다층 신경망을 설계하는 함수이다.

Cross Entropy

error에 민감하게 반응하여 가중치를 업데이트하는 Training기법이다.

마지막행렬의 오차값(e)에 Activation함수를 곱하지 않은 값으로 가중치를 업데이트 하는 것이 특징이다.

```
    %Back-propagation
    e     = d - y;
    delta = e;

    e3     = W4'*delta;
    delta3 = y3.*(1-y3).*e3;

    e2     = W3'*delta3;
    delta2 = y2.*(1-y2).*e2;

    e1     = W2'*delta2;
    delta1 = y1.*(1-y1).*e1;

    dW1 = alpha*delta1*x';
    W1 = W1 + dW1;

    dW2 = alpha*delta2*y1';
    W2 = W2 + dW2;

    dW3 = alpha*delta3*y2';
    W3 = W3 + dW3;

    dW4 = alpha*delta*y3';
    W4 = W4 + dW4;
  end

end
```

BackpropCE

BackpropCE 함수에서
마지막단의 델타값인

$$delta = \varphi'(v)e$$

가 아닌

$$delta = e$$

로 설정되는 것이 특징이다.

03

가중치 업데이트

CNN 1, 2

CNN 1 , 2 && MnistConv 1

+9 MnistConv1.m CNN1.m CNN2.m MnistConv2.m CNN3.m BackpropCE1.m fullyConne

```
clear all

Images = loadMNISTImages('/Users/zoocasso/Documents/MATLAB/t10k-images.idx3-ubyte'); % 파일 가져오기
Images = reshape (Images, 28, 28, []); % 28 x 28 x 10000
Labels = loadMNISTLabels('/Users/zoocasso/Documents/MATLAB/t10k-labels.idx1-ubyte');% 파일 가져오기 // 0~9
Labels(Labels == 0) = 10;

rng(1);

% Learning
%
W1 = 1e-2*randn([9 9 20]);
W5 = (2*rand(100, 2000) - 1);
Wo = (2*rand( 10,  100) - 1);

X = Images(:, :, 1:8000);
D = Labels(1:8000);

for epoch = 1:10
  epoch
  [W1, W5, Wo] = MnistConv1(W1, W5, Wo, X, D);
end

save('MnistConv1.mat');
```

CNN 1, 2

CNN 1 과 CNN2의 함수는 동일하게 MnistConv1함수를 불러와서 가중치를 저장한다.

save (' MnistConv1.mat ') ;
save (' MnistConv2.mat ') ;

CNN 3

CNN 3 && MnistConv 2

편집기 – /Users/zoocasso/Documents/MATLAB/CNN3.m

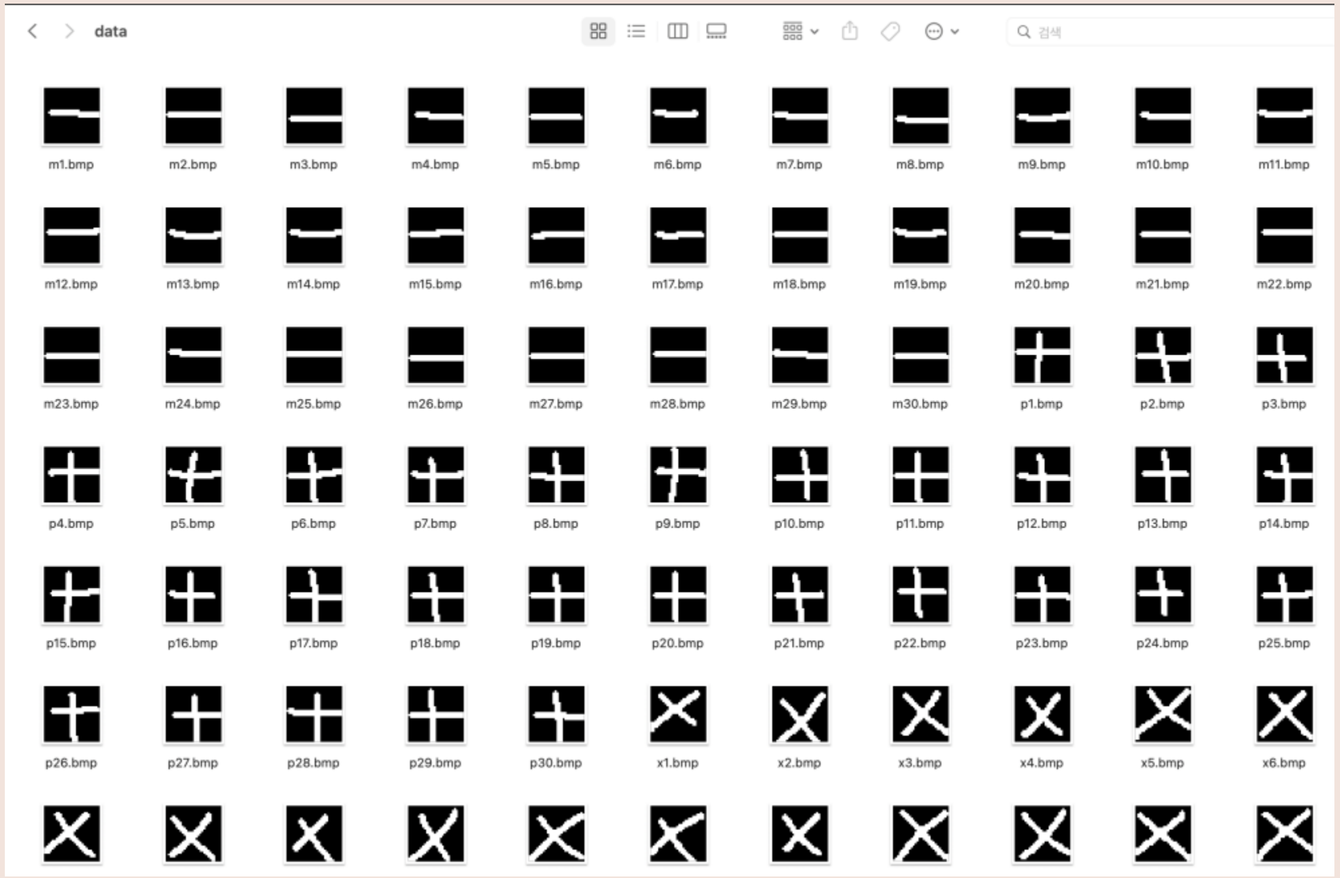
+9 | MnistConv1.m | CNN1.m | CNN2.m | MnistConv2.m | CNN3.m | BackpropCE1.m | fullyConnectedNN.m

```
clear all

for k = 1 : 30
    in(:,:,k) = im2double(imread(['/Users/zoocasso/Documents/MATLAB/data/p',num2str(k),'.bmp'])); % 파일 가져오기
    out(k,:) = 1;
end

for k = 1 : 30
    in(:,:,k+30) = im2double(imread(['/Users/zoocasso/Documents/MATLAB/data/m',num2str(k),'.bmp'])); % 파일 가져오기
    out(k+30,:) = 2;
end

for k = 1 : 30
    in(:,:,k+60) = im2double(imread(['/Users/zoocasso/Documents/MATLAB/data/x',num2str(k),'.bmp'])); % 파일 가져오기
    out(k+60,:) = 3;
end

idx =randperm(60,60);
X = in(:,:,idx);
D = out(idx,:);

% Learning
%
W1 = 1e-2*randn([9 9 20]);
W5 = (2*rand(100, 2000) - 1);
Wo = (2*rand( 3,  100) - 1);

for epoch = 1:10
    epoch
  [W1, W5, Wo] = MnistConv2(W1, W5, Wo, X, D);
end

save('MnistConv5.mat');
```

CNN 3

CNN3 은 CNN 1,2 와 입력값, 출력값이 다르므로 그에 맞게 설계된 MnistCon2함수를 불러와서 가중치를 저장한다.

save (' MnistConv3.mat ') ;



CNN 3

CNN3 의 INPUT값은
28x28 비트맵으로 작성하였다.

NN

NN && BackpropCE1

```
%Input 300x23
X = [
    1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 1 + 1 = 2
    0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 2       3
    0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 3       4
    0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 4       5
    0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 5       6
    0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 6       7
    0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 7       8
    0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 8       9
    0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 0; % 9       10
    0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 1 0 0; % 0       1

    1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0; % 1 + 2 = 3
    0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0;

    1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0; % 1 + 3 = 4
    0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 1 0 0;

    1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0; % 1 + 4 = 5
    0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0;
    0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 0;
```

```
D = [
    0 0 0 0 0 0 1 0; % plus part
    0 0 0 0 0 0 1 1;
    0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 1;
    0 0 0 0 0 1 1 0;
    0 0 0 0 0 1 1 1;
    0 0 0 0 1 0 0 0;
    0 0 0 0 1 0 0 1;
    0 0 0 0 1 0 1 0;
    0 0 0 0 0 0 0 1;

    0 0 0 0 0 0 1 1;
    0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 1;
    0 0 0 0 0 1 1 0;
    0 0 0 0 0 1 1 1;
    0 0 0 0 1 0 0 0;
    0 0 0 0 1 0 0 1;
    0 0 0 0 1 0 1 0;
    0 0 0 0 1 0 1 1;
    0 0 0 0 0 0 1 0;

    0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 1;
    0 0 0 0 0 1 1 0;
    0 0 0 0 0 1 1 1;
    0 0 0 0 1 0 0 0;
    0 0 0 0 1 0 0 1;
    0 0 0 0 1 0 1 0;
    0 0 0 0 1 0 1 1;
    0 0 0 0 1 1 0 0;
    0 0 0 0 0 0 1 1;

    0 0 0 0 0 1 0 1;
    0 0 0 0 0 1 1 0;
    0 0 0 0 0 1 1 1;
    0 0 0 0 1 0 0 0;
    0 0 0 0 1 0 0 1;
    0 0 0 0 1 0 1 0;
    0 0 0 0 1 0 1 1;
    0 0 0 0 1 1 0 0;
    0 0 0 0 1 1 0 1;
    0 0 0 0 0 1 0 0;
```

NN

input 과 output 을 다음과 같이 일일이 모두 작성해서 넣었다.

```
%Weight Matrix (-1 ~ 1 in random)
W1 = 2*rand(20,23) - 1;
W2 = 2*rand(20,20) - 1;
W3 = 2*rand(20,20) - 1;
W4 = 2*rand(8,20) - 1;
for epoch = 1:10000
    epoch
    [W1, W2, W3, W4] = BackpropCE1(W1,W2,W3,W4,X,D);
end

save('NN.mat');
```

NN

NN은 BackpropCE1함수를
불러와서 가중치를 저장한다.

save('NN.mat');

7segment

7segment && BackpropCE2

```
%Input 49x8
X = [
    0 0 0 0 0 0 0 0;
    0 0 0 0 0 0 0 1;
    0 0 0 0 0 0 1 0;
    0 0 0 0 0 0 1 1;
    0 0 0 0 0 1 0 0;
    0 0 0 0 0 1 0 1;
    0 0 0 0 0 1 1 0;
    0 0 0 0 0 1 1 1;
    0 0 0 0 1 0 0 0;
    0 0 0 0 1 0 0 1;
    0 0 0 0 1 0 1 0;
    0 0 0 0 1 0 1 1;
    0 0 0 0 1 1 0 0;
    0 0 0 0 1 1 0 1;
    0 0 0 0 1 1 1 0;
    0 0 0 0 1 1 1 1;
    0 0 0 1 0 0 0 0;
    0 0 0 1 0 0 0 1;
    0 0 0 1 0 0 1 0;
    1 0 0 0 0 0 0 1;
    1 0 0 0 0 0 1 0;
    1 0 0 0 0 0 1 1;
    1 0 0 0 0 1 0 0;
    1 0 0 0 0 1 0 1;
    1 0 0 0 0 1 1 0;
    1 0 0 0 0 1 1 1;
    1 0 0 0 1 0 0 0;
    1 0 0 0 1 0 0 1;
    0 0 0 1 0 1 0 0;
    0 0 0 1 0 1 0 1;
    0 0 0 1 1 0 0 0;
    0 0 0 1 1 0 0 1;
    0 0 0 1 1 0 1 1;
    0 0 0 1 1 1 0 0;
    0 0 0 1 1 1 1 0;
    0 0 1 0 0 0 0 0;
    0 0 1 0 0 0 1 1;
    0 0 1 0 0 1 0 0;
    0 0 1 0 1 0 0 0;
    0 0 1 0 1 0 1 0;
    0 0 1 0 1 1 0 1;
    0 0 1 1 0 0 0 0;
```

```
%Correct Output 49x14
D = [
    1 1 1 1 1 1 0 1 1 1 1 1 1 0;
    1 1 1 1 1 1 0 0 1 1 0 0 0 0;
    1 1 1 1 1 1 0 1 1 0 1 1 0 1;
    1 1 1 1 1 1 0 1 1 1 1 0 0 1;
    1 1 1 1 1 1 0 0 1 1 0 0 1 1;
    1 1 1 1 1 1 0 1 0 1 1 0 1 1;
    1 1 1 1 1 1 0 1 0 1 1 1 1 1;
    1 1 1 1 1 1 0 1 1 1 0 0 1 1;
    1 1 1 1 1 1 0 1 1 1 1 1 1 1;
    1 1 1 1 1 1 0 1 1 1 0 0 1 1;
    0 1 1 0 0 0 0 1 1 1 1 1 1 0;
    0 1 1 0 0 0 0 0 1 1 0 0 0 0;
    0 1 1 0 0 0 0 1 1 0 1 1 0 1;
    0 1 1 0 0 0 0 1 1 1 1 0 0 1;
    0 1 1 0 0 0 0 0 1 1 0 0 1 1;
    0 1 1 0 0 0 0 1 0 1 1 0 1 1;
    0 1 1 0 0 0 0 1 0 1 1 1 1 1;
    0 1 1 0 0 0 0 1 1 1 0 0 1 0;
    0 1 1 0 0 0 0 1 1 1 1 1 1 1;
    0 0 0 0 0 0 1 0 1 1 0 0 0 0;
    0 0 0 0 0 0 1 1 1 0 1 1 0 1;
    0 0 0 0 0 0 1 1 1 1 1 0 0 1;
    0 0 0 0 0 0 1 0 1 1 0 0 1 1;
    0 0 0 0 0 0 1 1 0 1 1 0 1 1;
    0 0 0 0 0 0 1 1 0 1 1 1 1 1;
    0 0 0 0 0 0 1 1 1 1 0 0 1 0;
    0 0 0 0 0 0 1 1 1 1 1 1 1 1;
    0 0 0 0 0 0 1 1 1 1 0 0 1 1;
    1 1 0 1 1 0 1 1 1 1 1 1 1 0
    1 1 0 1 1 0 1 0 1 1 0 0 0 0
    1 1 0 1 1 0 1 0 1 1 0 0 1 1
    1 1 0 1 1 0 1 1 0 1 1 0 1 1
    1 1 0 1 1 0 1 1 1 1 0 0 1 0
    1 1 0 1 1 0 1 1 1 1 1 1 1 1
    1 1 1 1 0 0 1 1 1 1 1 1 1 0
    1 1 1 1 0 0 1 1 1 0 1 1 0 1
    1 1 1 1 0 0 1 1 0 1 1 0 1 1
    1 1 1 1 0 0 1 1 0 1 1 1 1 1
    0 1 1 0 0 1 1 1 1 1 1 1 1 0
    0 1 1 0 0 1 1 1 1 0 1 1 0 1
    0 1 1 0 0 1 1 1 0 1 1 0 1 1
    0 1 1 0 0 1 1 1 1 1 1 1 1 1
```

7segment

input 과 output 을 다음과 같이
일일이 모두 작성해서 넣었다.

7segment

```
%Weight Matrix (-1 ~ 1 in random)
W1 = 2*rand(20,8) - 1;
W2 = 2*rand(20,20) - 1;
W3 = 2*rand(20,20) - 1;
W4 = 2*rand(14,20) - 1;
for epoch = 1:10000
    epoch
    [W1, W2, W3, W4] = BackpropCE2(W1,W2,W3,W4,X,D);
end

save('sevenSegment.mat');
```

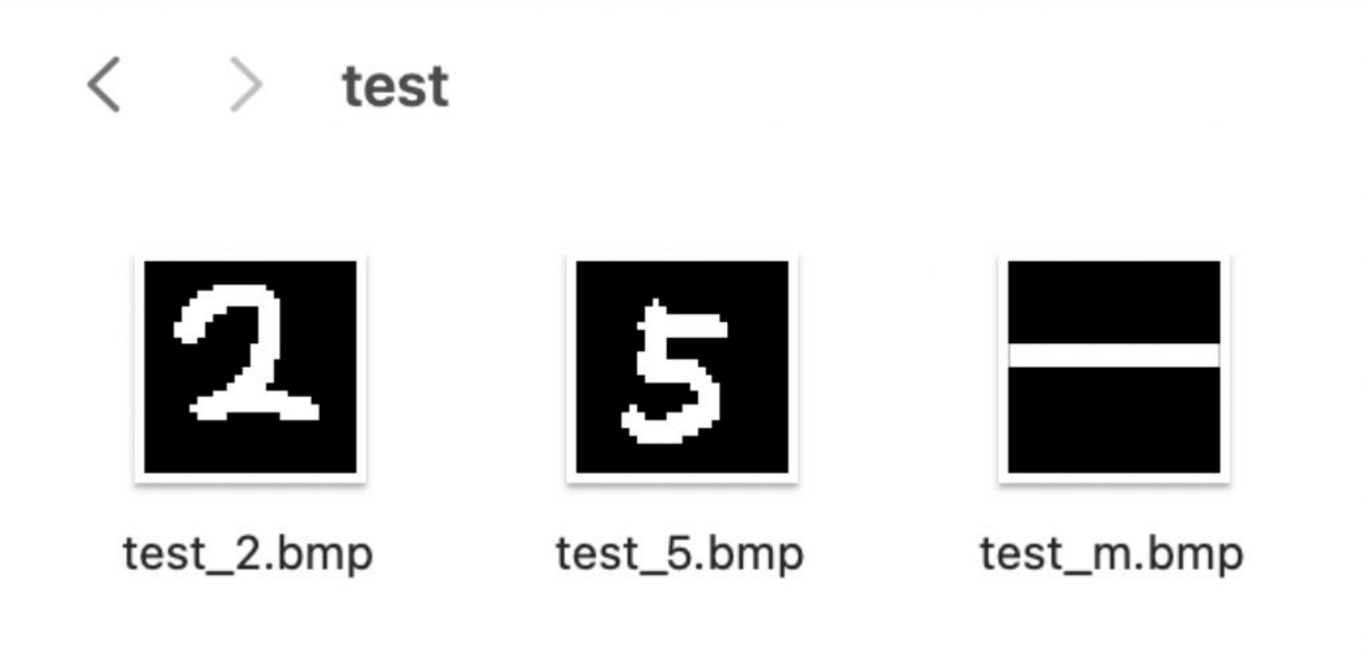
NN은 BackpropCE1함수를
불러와서 가중치를 저장한다.

save(' sevenSegment.mat');

03

결과

INPUT	image 3개	23bit one-hot-encoding	8bit 이진수
	CNN (0~9)	**NN** (0~9)	**7segment**
OUTPUT	23bit one-hot-encoding	8bit 이진수	14bit 이진수



테스트 함수

직접 그린 28x28 비트맵을 INPUT 으로 해서 테스트해보자.

CNN1 = 2,
CNN2 = 5,
CNN3 = '-' 으로 출력은 -3 이다.

image
2

→

10bit
0100000000

CNN1

```
%CNN1
load("MnistConv1.mat");
%input image
X(:,:,1) = im2double(imread(['/Users/zoocasso/Documents/MATLAB/test/test_2.bmp']));
x = X(:, :, 1);                    % Input,          28x28

  y1 = Conv(x, W1);                  % Convolution,  20x20x20
  y2 = ReLU(y1);                     %
  y3 = Pool(y2);                     % Pool,         10x10x20
  y4 = reshape(y3, [], 1);           %                   2000
  v5 = W5*y4;                        % ReLU,              360
  y5 = ReLU(v5);                     %
  v  = Wo*y5;                        % Softmax,            10
  y  = Softmax(v);                   %
  cnn1 = [y]'
```

image 를 INPUT으로 one-hot-encoding 10bit를 OUTPUT으로 하는 CNN이다.

test_2.bmp을 INPUT으로 했을때 2가 정상적으로 출력된다.

명령 창

```
>> test

cnn1 =

    0.0000    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000
```

image
5

→

10bit
0000100000

CNN2

```
%CNN2
load('MnistConv2.mat');
%input image
X(:,:,1) = im2double(imread(['/Users/zoocasso/Documents/MATLAB/test/test_5.bmp']));
x = X(:, :, 1);                    % Input,           28x28

  y1 = Conv(x, W1);                  % Convolution,  20x20x20
  y2 = ReLU(y1);                     %
  y3 = Pool(y2);                     % Pool,         10x10x20
  y4 = reshape(y3, [], 1);           %                   2000
  v5 = W5*y4;                        % ReLU,              360
  y5 = ReLU(v5);                     %
  v  = Wo*y5;                        % Softmax,            10
  y  = Softmax(v);                   %
  cnn2 = [y]'
```

image 를 INPUT으로 one-hot-encoding 10bit를 OUTPUT으로 하는 CNN이다.

test_5.bmp를 INPUT으로 했을때 5가 정상적으로 출력된다.

명령 창

```
>> test

cnn2 =

    0.0000    0.0000    0.0000    0.0000    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000
```

image

—

→

3bit

010

CNN3

```
%CNN3
load('MnistConv3.mat');
%input image
X(:,:,1) = im2double(imread(['/Users/zoocasso/Documents/MATLAB/test/test_m.bmp']));
x = X(:, :, k);                    % Input,           28x28

  y1 = Conv(x, W1);                  % Convolution,  20x20x20
  y2 = ReLU(y1);                     %
  y3 = Pool(y2);                     % Pool,         10x10x20
  y4 = reshape(y3, [], 1);           %                   2000
  v5 = W5*y4;                        % ReLU,              360
  y5 = ReLU(v5);                     %
  v  = Wo*y5;                        % Softmax,            10
  v(isnan(v)) = 0;
  y  = Softmax(v);                   %
  y(isnan(y))=0;
  cnn3 = [y]';
```

image 를 INPUT으로 one-hot-encoding 3bit를 OUTPUT으로 하는 CNN이다.

test_m.bmp를 INPUT으로 했을때 ' — ' 가 정상적으로 출력된다.

명령 창

```
>> test

cnn3 =

    0.0000    1.0000    0.0000
```

NN

10bit	10bit	3bit
0100000000	0000100000	010

```
NNinput = horzcat(cnn1,cnn2,cnn3);
```

NNinput =

23bit
0100000000
0000100000
010

cnn1, cnn2, cnn3을 하나의 배열로 합치기 위해 horzcat 함수를 사용한다.

NNinput = cnn1 + cnn2 + cnn3

명령 창

```
>> test

NNinput =

  1 ~ 12번 열

    0.0000    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000

  13 ~ 23번 열

    0.0000    0.0000    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    1.0000    0.0000
```

23bit

0100000000

0000100000

010

→

8bit

10000011

NN

```
%one hot encoding -> 8bit
load('NN.mat');
%input 13bit
NNinput = horzcat(cnn1,cnn2,cnn3);
x = NNinput(1,:)'
        v1 = W1*x;
        y1 = Sigmoid(v1);

        %2nd Hidden Layer
        v2 = W2*y1;
        y2 = Sigmoid(v2);

        %3rd Hidden Layer
        v3 = W3*y2;
        y3 = Sigmoid(v3);

        %Output Layer
        v = W4*y3;
        y = Sigmoid(v)';
        eightBit= y
```

one-hot-encoding 23bit를 INPUT으로 8bit 이진수를 OUTPUT으로 하는 CNN이다.

명령 창

```
>> test

eightBit =

    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    1.0000    1.0000
```

7segment

8bit

10000011

→

14bit

0000001

1111001

8bit 이진수를 INPUT으로
14bit 이진수를 OUTPUT으로 하는
CNN이다.

```
%7segment
load('sevenSegment.mat');
%input 8bit
%1st Hidden Layer
        x = eightBit';
        v1 = W1*x;
        y1 = Sigmoid(v1);

        %2nd Hidden Layer
        v2 = W2*y1;
        y2 = Sigmoid(v2);

        %3rd Hidden Layer
        v3 = W3*y2;
        y3 = Sigmoid(v3);

        %Output Layer
        v = W4*y3;
        y = Sigmoid(v)'
```

명령 창

```
>> test

y =

  1 ~ 12번 열

    0.0000    0.0000    0.0002    0.0000    0.0000    0.0000    1.0000    1.0000    1.0000    1.0000    0.9999    0.0001

  13 ~ 14번 열

    0.0001    0.9999
```